bcrpyt

bcrpyt ist ein Algorithmus, der 1999 vorgestellt wurde und als sicher gilt:

- bcrypt ist langsam!
  Warum ist das ein Vorteil?
- Genauer: bcrypt‘s Komplexität ist über einen Kostenfaktor einstellbar
- Automatische Erstellung von Salts
- bcrypt ist so konstruiert, dass es nur wenig von Hardware-Beschleunigung z.B. durch GPUs profitiert
- Welche Informationen über das Passwort müssen Sie in der Datenbank abspeichern?

```
import org.mindrot.jbcrypt.BCrypt;

public class Main {
    public static void main(String[] args) {
        String password = args[0];
        String salt;
        String hashed;

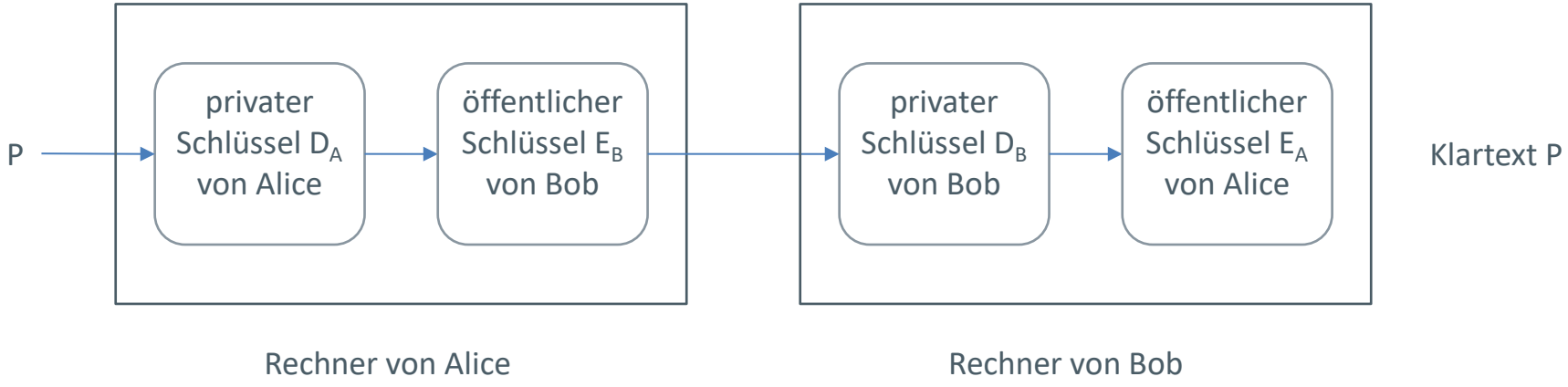
        for (int i=0;i<10;i++) {
            salt = BCrypt.gensalt(5);
            hashed = BCrypt.hashpw(password, salt);
            System.out.println("Salt= " + salt + ", hash= " +
hashed );
        }
    }
}
```

Beispiel mit jbcrypt

Digitale Signaturen

Wie kann man sicher sein, dass eine erhaltene Nachricht „echt“ ist?

a) Empfänger kann die Identität des Senders überprüfen (**Authentizität**)
b) Der Sender kann den Inhalt der Nachricht nicht verleugnen (**Nichtabstreitbarkeit**)
c) Die Nachricht wurde auf dem Übertragungsweg nicht verändert (**Integrität**)
d) Der Empfänger kann die Nachricht nicht selbst hergestellt haben.



$$E_A(D_B(E_B(D_A(P))) = P$$

```
import javax.crypto.*;
import java.security.*;
```

Java Demo

Generierung eines Schlüsselpaares:

```
private static KeyPair generateKeyPair() throws NoSuchAlgorithmException {
    KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
    keyGen.initialize(2048);
    return keyGen.generateKeyPair();
}
```

Verschlüsseln mit private und public keys:

```
byte[] E1 = encrypt(Msg, aliceKeyPair.getPrivate());
byte[] E2 = encrypt(E1, bobKeyPair.getPublic());
```

Entschlüsseln mit private und public keys:

```
byte[] D1 = decrypt(Msg, bobKeyPair.getPrivate());
byte[] D2 = encrypt(D1, aliceKeyPair.getPublic());
```

Zertifikate

Woher weiß Bob, dass der öffentliche Schlüssel von Alice wirklich Alice gehört, wenn er Alice vielleicht gar nicht kennt?

Ein Zertifikat beweist die Echtheit eines öffentlichen Schlüssels.

Ich zertifiziere, dass der öffentliche Schlüssel `3498AFEB3ED00`
Alice
Hauptstrasse 4711
77777 Musterstadt
gehört

SHA-256 Hash des obigen Zertifikats signiert mit dem privaten Schlüssel der Zertifizierungsstelle

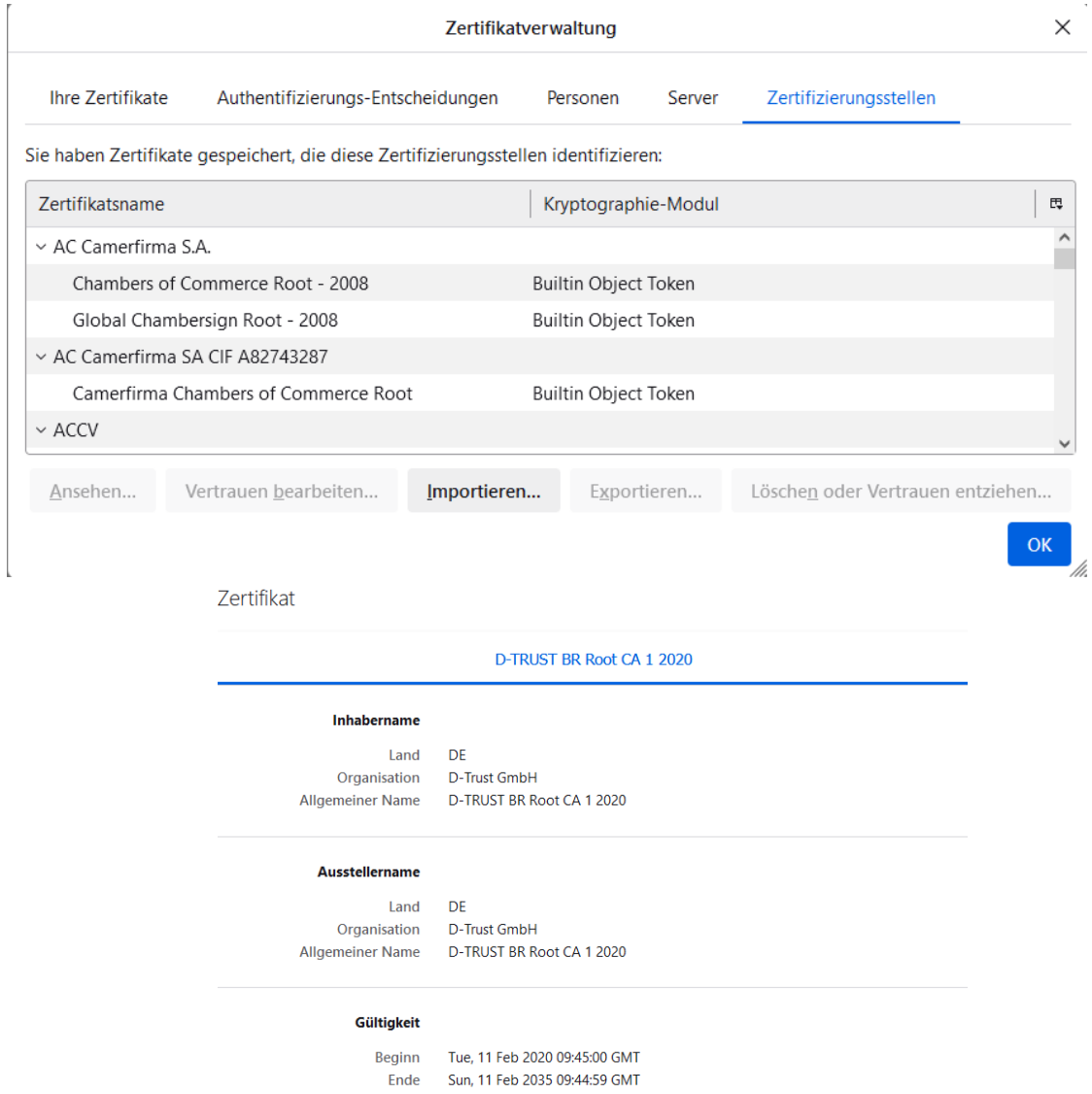
Daraus ergibt sich die nächste Frage: Wer ist „Ich“ ?
Wieso kann Bob dieser Person trauen?

Public-Key Infrastructure

- Viele Root-CA‘s (oberste Zertifizierungsstellen),
- die wiederum andere CA‘s zertifizieren,
- usw.

- In Firefox sind z.B. bereits ~250 Zertifikate vorinstalliert.

- Weitere Sicherheitsmechanismen:
  - Typischerweise Gültigkeit von 3 Monaten – n Jahre
  - Certificate Revocation Lists (Sperrlisten)



Authentifizierung

- Ziele:
  - Eindeutige Feststellung der Identität
  - Abwehr von Identitätsdiebstahl

- Das zu authentifizierende Subjekt ist entweder
  - Ein Mensch
  - Ein Gerät (Server, Laptop, …)
  - Ein Dienst (Online-Shop, Online-Banking, …)



Mechanismen zur Authentifizierung

Authentifizierung geschieht durch

- **Wissen** („ich weiß das Kennwort“)
- **Besitz** (USB-Token, SIM-Karte im Mobiltelefon, …)
- **Merkmale** (Fingerabdruck, Iris, …)

Warum ist ein Mechanismus alleine problematisch?

Mehrfaktor-Authentifizierung:

- Kombination mehrerer Mechanismen ist erforderlich.
- Beispiel:
  - PIN der SIM-Karte + SIM-Karte
  - eCampus Passwort + TAN

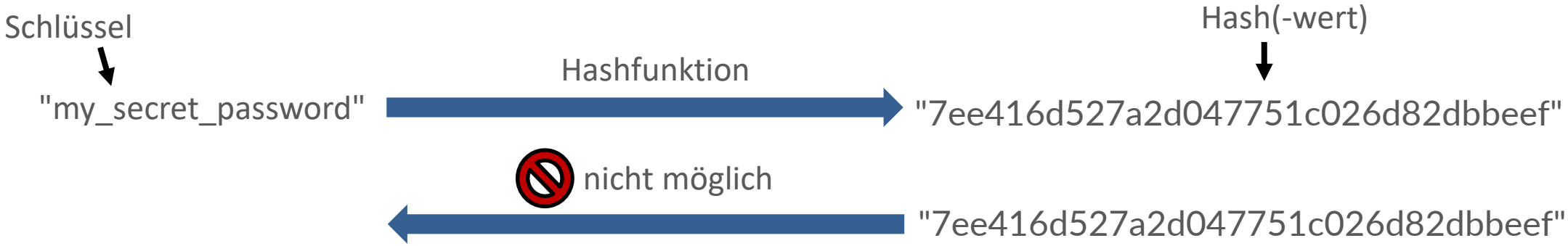


“This site wants a two-factor authentication. A retina scan and a urine sample.”

Authentifizierung durch Wissen

Andere Möglichkeiten:
Challenge-Response, SSO, …
-> INF2205 Einführung IT-Security

- Immer noch am häufigsten: Passwörter !
- Was sind die Schwächen von Passwörtern?
  - Oft leicht zu erraten oder auszuprobieren
  - Passwörter könnten unverschlüsselt übertragen werden
  - Benutzer verwenden viele Passwörter mehrmals (besonders, wenn sie ihr Passwort periodisch wechseln müssen).
  - …
- UNIX Passwort-Verfahren
  - In /etc/shadow wird nur der Hash des Passwortes gespeichert
  - Von einem eingegebenen Passwort wird auch der Hash berechnet und verglichen



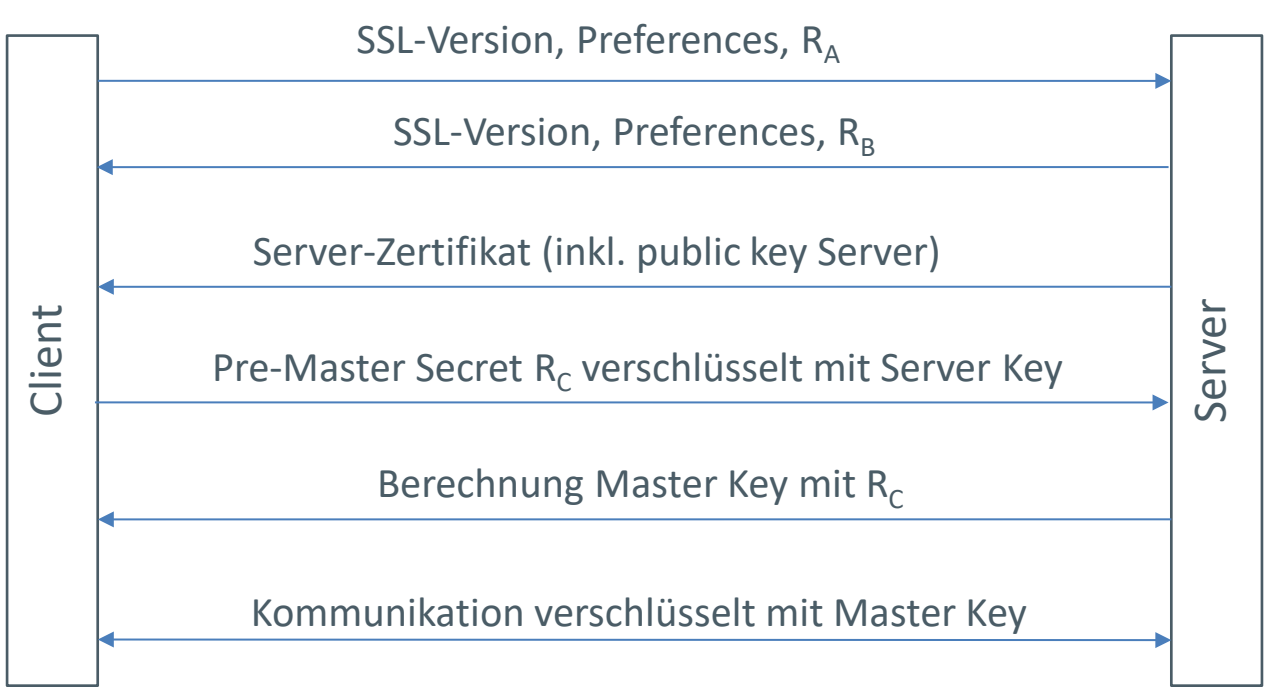
TLS (SSL)



Ziel: Der Server (!) authentifiziert sich gegenüber dem Client und beide können miteinander verschlüsselt kommunizieren.

- **SSL** : Secure Sockets Layer wurde bis 1999 verwendet
- **TLS**: (Transport Level Security) v1.0 (1999) war mit SSL v3 identisch. Seitdem wird nur noch der Name TLS benutzt.
  Aktuelle Version: 1.3.
- TLS ist ein „Add-on“ in OSI-Schicht 5, um existierende Protokolle wie z.B. http „sicher“ zu machen.

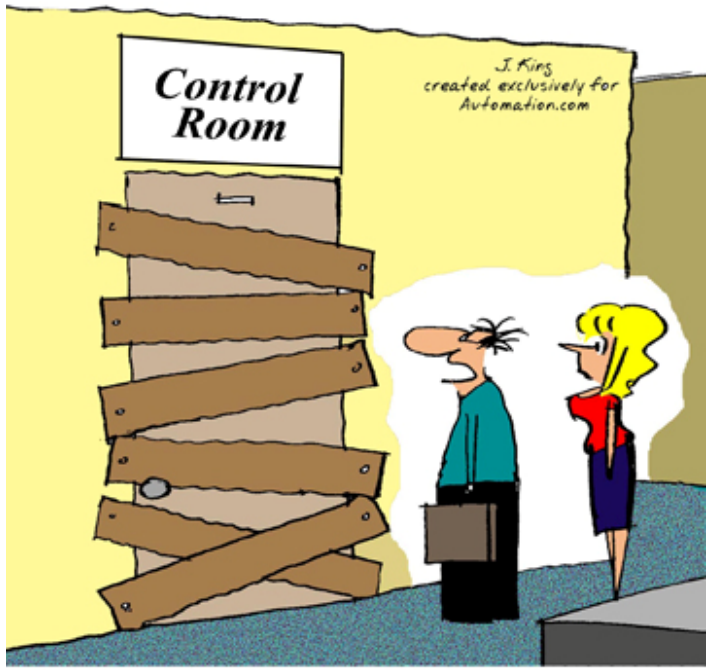
Ablauf SSL / TLS



A) Handshake-Protokoll, um **asymmetrisch** einen symmetrischen Schlüssel auszuhandeln.

B) Kommunikation mit **symmetrischer** Verschlüsselung

Zugriffskontrolle



"When I told management that we needed to increase security around our control systems, this is NOT what I had in mind..."

Autorisierung

Ziel: Festzulegen und Sicherstellen, welche Benutzer oder Systeme Zugriff auf spezifische Ressourcen, Daten oder Funktionen innerhalb eines IT-Systems haben.

„Einfache Variante“: Diskretionäre Zugriffskontrolle (**DAC**)

- Eigentümer entscheidet, wer auf was zugreifen darf.
- Einfach umsetzbar mit einer Matrix
  Subjekte (z.B. Benutzer) x Objekte (z.B. Daten, Dateien, …)

- An welches Rechtemodell aus PI2 erinnert Sie das?
- Unix-Rechtesystem auf Dateien (rwx)



Rollenbasierte Zugriffskontrolle

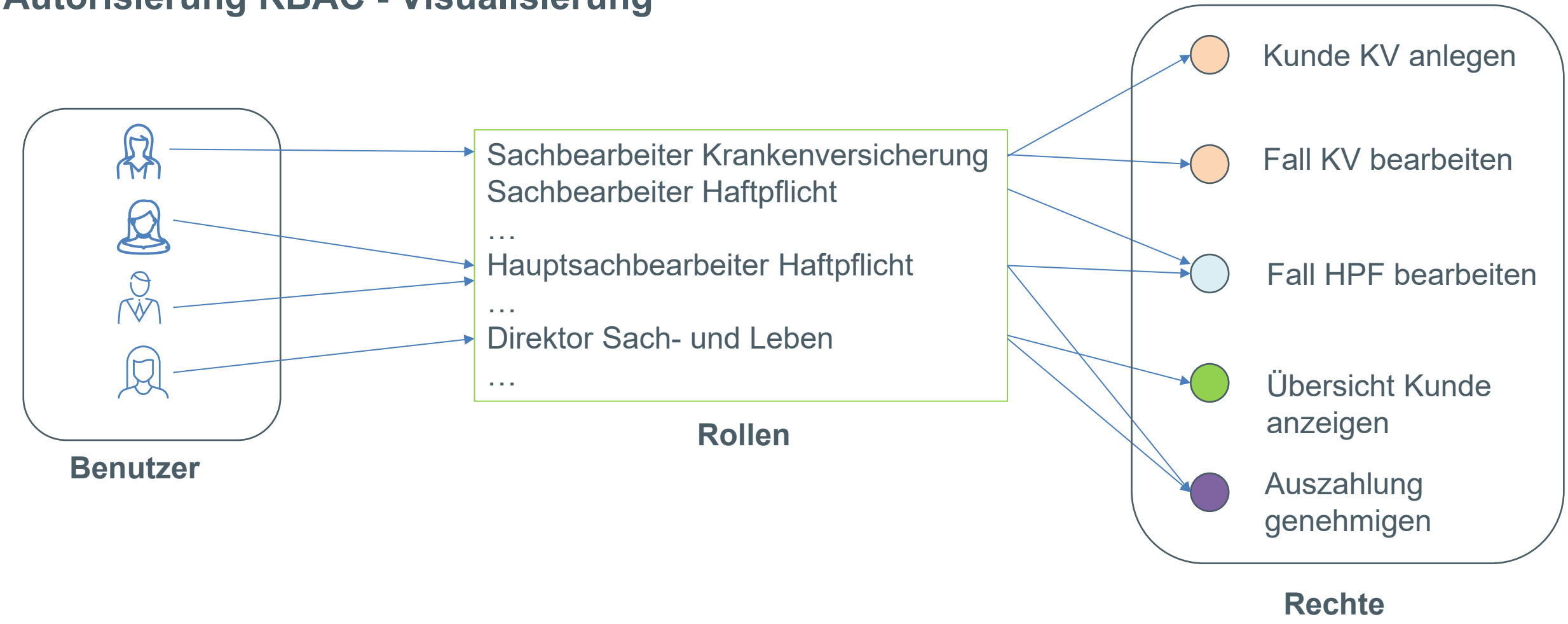
"Rollen-Rechte-Matrix" (role-based access control, **RBAC**):

- Den Benutzern werden ihre Rollen zugeordnet:
  Beispiel im SB-Dienst: Studierende(r), Prüfer
- Den Objekten werden Rollen mit Rechten zugeordnet:
  Beispiel: Rolle Prüfer darf Objekt "Benotung" lesen und schreiben;
  Rolle Studierende(r) darf "Benotung" nur lesen.

Reicht das?

- Im letzten Beispiel hätte jede(r) Studierende(r) Leserechte auf allen Benotungen.
- Daher müssen die Rollen und Rechte i.A. durch Policies ergänzt werden:
  Rolle Studierender darf "Benotung" lesen, wenn UserID.Matrikelnummer == Benotung.Matrikelnummer

Autorisierung RBAC - Visualisierung



RBAC ist oft an Organisationsstruktur des Unternehmens orientiert.

Vor- und Nachteile

DAC

+ Intuitiv & flexibel
+ Einfach zu implementieren
+ Standard in (fast) allen Betriebssystemen

- Riesige Tabellen, skaliert eher schlecht
- Vergeben und Zurücknehmen von Rechten komplex und führt oft zu Inkonsistenzen

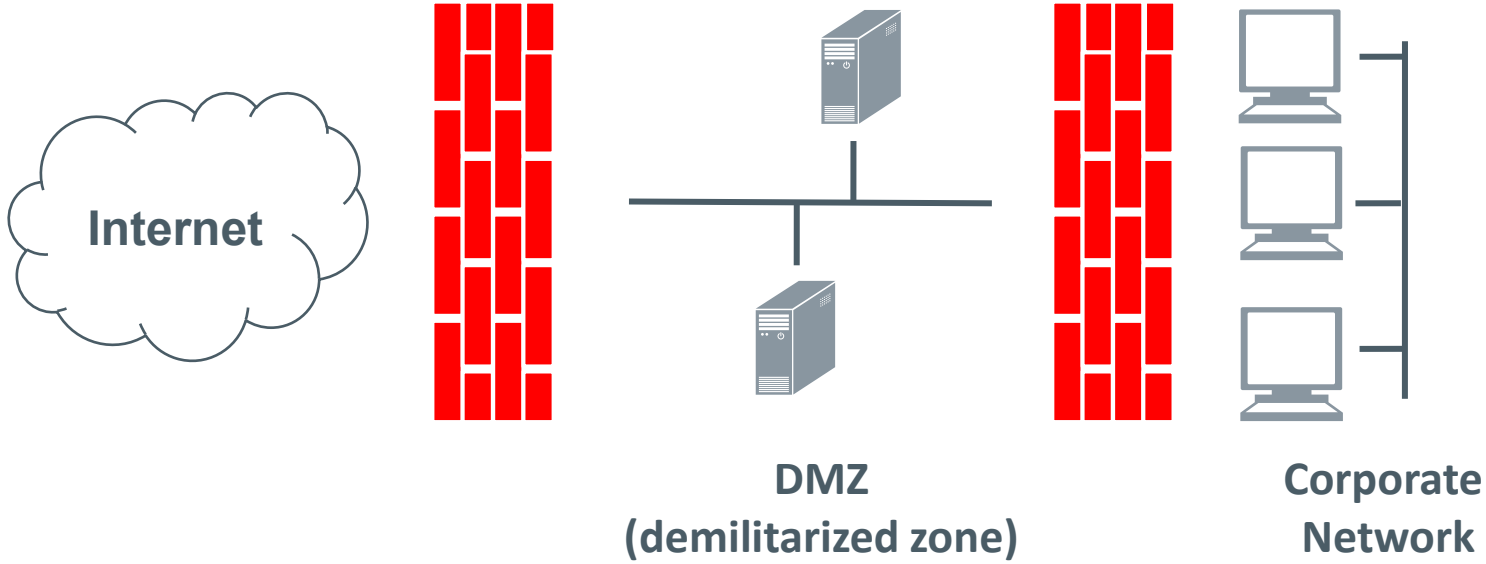
RBAC

+ Skaliert sehr gut
+ Direktes Nachbilden der Organisationsstruktur
+ De-Facto Standard in Unternehmenssoftware wie z.B. SAP
+ Die Abbildung Rollen -> Rechte ändert sich selten, während sich Rollenmitgliedschaften öfter ändern

- Hoher Aufwand für die Pflege der Rollen und der Mitarbeiterzuordnung

Firewalls

- Typischerweise (hoffentlich!) haben Firmen zwei Firewalls zwischen ihrem Corporate Network und dem Internet:



- Firewalls filtern Pakete und können Ports blockieren.

Beispiele für Firewall Filtering and Blocking

Regel	Firewall Einstellung (Filter)
Keine eingehenden Verbindungen außer zum Web Server	Blockieren aller eingehenden TCP und UDP Pakete außer 134.176.243.32:80
Eintreffende Mail zulassen	Erlauben aller eingehenden Pakete zu Mail-Gateway IP und Port 25
…	

Warum sollte man in den Filtern nur IP-Adressen verwenden und keine Hostnamen?

Intrusion Detection Systems (IDS)

- IDS sind erlaubte Abhörgeräte im Netzwerk
- Lauschen typischerweise nur auf eingehende Pakete
- Entscheiden, ob ein Paket Malware enthält, indem sie die Nutzdaten gegen bekannte Viren/Trojaner/… vergleichen
- Senden eines Alarms
- Nutzlos gegen neue („zero-day“) Angriffe – warum?

- Problem:
  - Zu sensitiv: Zu viele Alarme
  - Zu wenig sensitiv: Malware nicht erkannt

